SCS 2312
Computational Models and Programming Language Concepts

Describe a Language

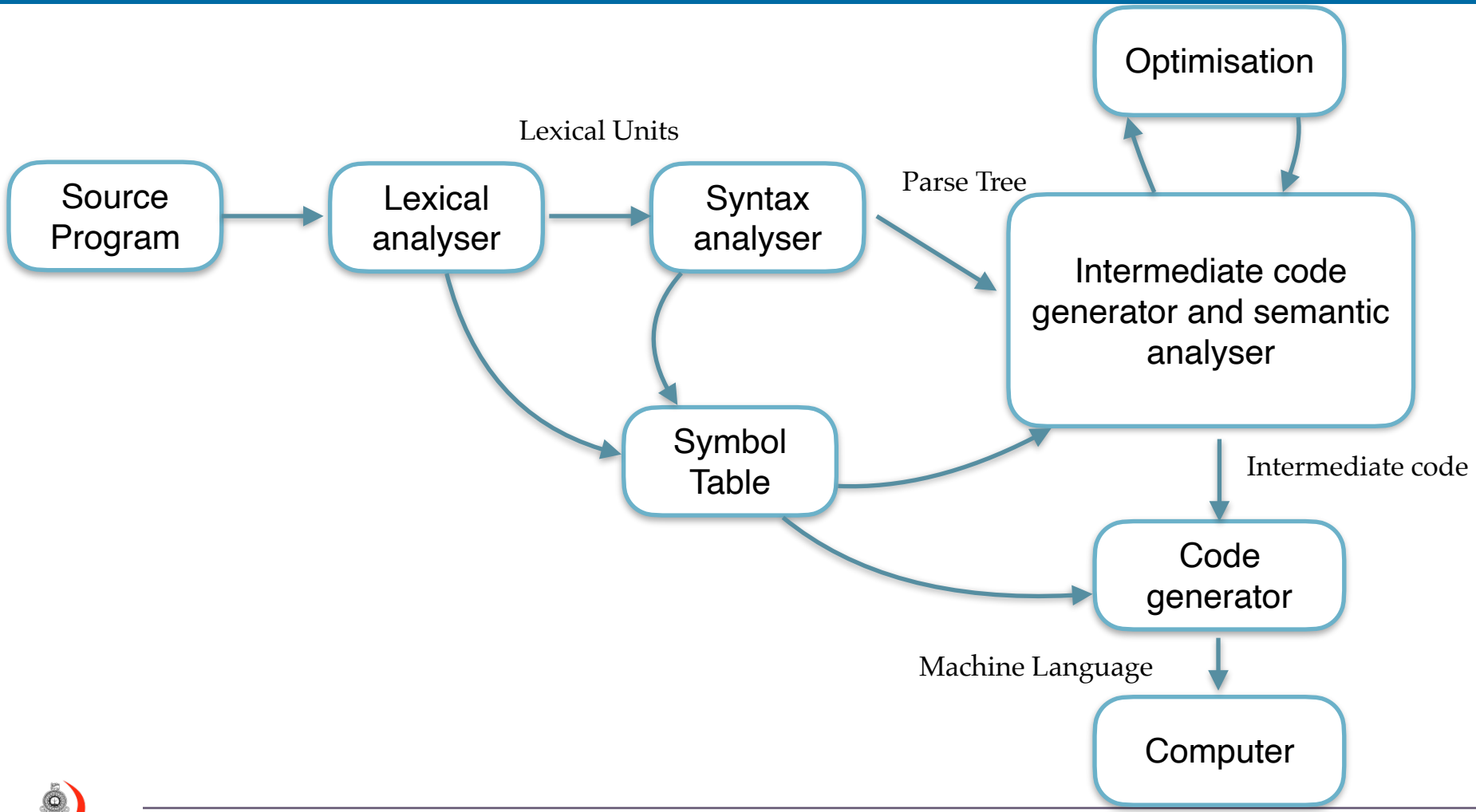
Tharindu Wijethilake
tnb@ucsc.cmb.ac.lk

UNIVERSITY OF COLOMBO SCHOOL OF COMPUTING

Plan for the day

- Why Languages
- Why Grammars
- Describe a Language

Compilation



Compilation

- A compiler is a special program that takes the entire **source code** of the program and translates it into machine code before the program is executed.

A simple C code

```c
#include <stdio.h>

int main(){
    printf("%s", "Hello World \n");
    return 0;
}
```

```
    .section    __TEXT,__text,regular,pure_instructions
    .build_version macos, 10, 15, 6 sdk_version 10, 15, 6
    .globl  _main                   ## -- Begin function main
    .p2align    4, 0x90
_main:                                  ## @main
    .cfi_startproc
## %bb.0:
    pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset %rbp, -16
    movq    %rsp, %rbp
    .cfi_def_cfa_register %rbp
    subq    $16, %rsp
    movl    $0, -4(%rbp)
    leaq    L_.str(%rip), %rdi
    leaq    L_.str.1(%rip), %rsi
    movb    $0, %al
    callq   _printf
    xorl    %ecx, %ecx
    movl    %eax, -8(%rbp)          ## 4-byte Spill
    movl    %ecx, %eax
    addq    $16, %rsp
    popq    %rbp
    retq
    .cfi_endproc
                                        ## -- End function
    .section    __TEXT,__cstring,cstring_literals
L_.str:                                 ## @.str
    .asciz  "%s"

L_.str.1:                               ## @.str.1
    .asciz  "Hello World \n"

.subsections_via_symbols
```

```
Disassembly of section __TEXT,__text:

0000000100003f20 _fun1:
100003f20: 55                          pushq   %rbp
100003f21: 48 89 e5                    movq    %rsp, %rbp
100003f24: c7 05 e2 40 00 00 11 00 00 00       movl    $17, 16610(%rip)
100003f2e: b8 03 00 00 00              movl    $3, %eax
100003f33: 5d                          popq    %rbp
100003f34: c3                          retq
100003f35: 66 2e 0f 1f 84 00 00 00 00 00       nopw    %cs:(%rax,%rax)
100003f3f: 90                          nop

0000000100003f40 _main:
100003f40: 55                          pushq   %rbp
100003f41: 48 89 e5                    movq    %rsp, %rbp
100003f44: 48 83 ec 10                 subq    $16, %rsp
100003f48: c7 45 fc 00 00 00 00        movl    $0, -4(%rbp)
100003f4f: 8b 05 bb 40 00 00           movl    16571(%rip), %eax
100003f55: 89 45 f8                    movl    %eax, -8(%rbp)
100003f58: e8 c3 ff ff ff              callq   -61 <_fun1>
100003f5d: 8b 4d f8                    movl    -8(%rbp), %ecx
100003f60: 01 c1                       addl    %eax, %ecx
100003f62: 89 0d a8 40 00 00           movl    %ecx, 16552(%rip)
100003f68: 8b 35 a2 40 00 00           movl    16546(%rip), %esi
100003f6e: 48 8d 3d 35 00 00 00        leaq    53(%rip), %rdi
100003f75: b0 00                       movb    $0, %al
100003f77: e8 0e 00 00 00              callq   14 <dyld_stub_binder+0x100003f8a>
100003f7c: 31 c9                       xorl    %ecx, %ecx
100003f7e: 89 45 f4                    movl    %eax, -12(%rbp)
100003f81: 89 c8                       movl    %ecx, %eax
100003f83: 48 83 c4 10                 addq    $16, %rsp
100003f87: 5d                          popq    %rbp
100003f88: c3                          retq

Disassembly of section __TEXT,__stubs:

0000000100003f8a __stubs:
100003f8a: ff 25 70 40 00 00           jmpq    *16496(%rip)

Disassembly of section __TEXT,__stub_helper:

0000000100003f90 __stub_helper:
100003f90: 4c 8d 1d 71 40 00 00        leaq    16497(%rip), %r11
100003f97: 41 53                       pushq   %r11
100003f99: ff 25 61 00 00 00           jmpq    *97(%rip)
100003f9f: 90                          nop
100003fa0: 68 00 00 00 00              pushq   $0
100003fa5: e9 e6 ff ff ff              jmp -26 <__stub_helper>
```

Question

- How can we formally, precisely, and unambiguously describe the potentially infinite set of valid programs finitely?

Program

- Written in a language.
- Language has **Grammar.**

Formal Grammar

- A formal grammar provides a finite set of rules (the grammar) that can recursively generate all valid sequences (programs) in the language, and only those valid sequences.
- The power of grammar comes from **recursion**.

Programming Languages

- **Syntax** of a programming language is the form of its expressions, statements, and program units.
- **Semantics** is the meaning of those expressions, statements, and program units.

Describe a Language

- Need a formal method to describe the language.
  - Context-Free Grammar (CFG)
    - A formal, mathematical concept
  - Backus-Naur Form (BNF)
    - A notation or a descriptive metalanguage.

Context-Free Grammar (CFG)

- Noam Chomsky
- A Context-free grammar is a formal system used to describe the syntax of a language.
- Define the syntactic structure of programming languages.
- Four main components
  - Terminal symbols (Σ)
  - Nonterminal symbols (V)
  - Production rules (P)
  - Start symbol (S)

Backus-Naur Form (BNF)

- John Backus and Peter Naur
- A BNF grammar is a set of production rules that define how to construct valid strings in a language
- The concrete notation or syntax for writing down the CFG

Grammar

<program> → **begin** <stmt_list> **end**

<stmt_list> → <stmt>
| <stmt> ; <stmt_list>

<stmt> → <var> = <expression>

<var> → A | B | C

<expression> → <var> + <var>
| <var> – <var>
| <var>

Concepts of Programming Languages, Robert W. Sebesta

Very Simple Example

<digit> → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

<number> → <digit>

| <digit> <number>

Derivation

<number> => <digit>

=> 2

<number> => <digit> <number>

=> 2 <digit>

=> 2 4

Grammar

<program> → **begin** <stmt_list> **end**

<stmt_list> → <stmt>
| <stmt> ; <stmt_list>

<stmt> → <var> = <expression>

<var> → A | B | C

<expression> → <var> + <var>
| <var> – <var>
| <var>

Concepts of Programming Languages, Robert W. Sebesta

Grammar

- Simple assignment statement

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <id> + <expr>
| <id> * <expr>
| (<expr>)
| <id>

```
A = B * ( A + C )
```

Concepts of Programming Languages, Robert W. Sebesta

Grammar

- Simple assignment statement

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <id> + <expr>
| <id> * <expr>
| (<expr>)
| <id>

<assign> => <id> = <expr>
=> A = <expr>
=> A = <id> * <expr>
=> A = B * <expr>
=> A = B * (<expr>)
=> A = B * (<id> + <expr>)
=> A = B * (A + <expr>)
=> A = B * (A + <id>)
=> A = B * (A + C)

```
A = B * ( A + C )
```

Concepts of Programming Languages, Robert W. Sebesta

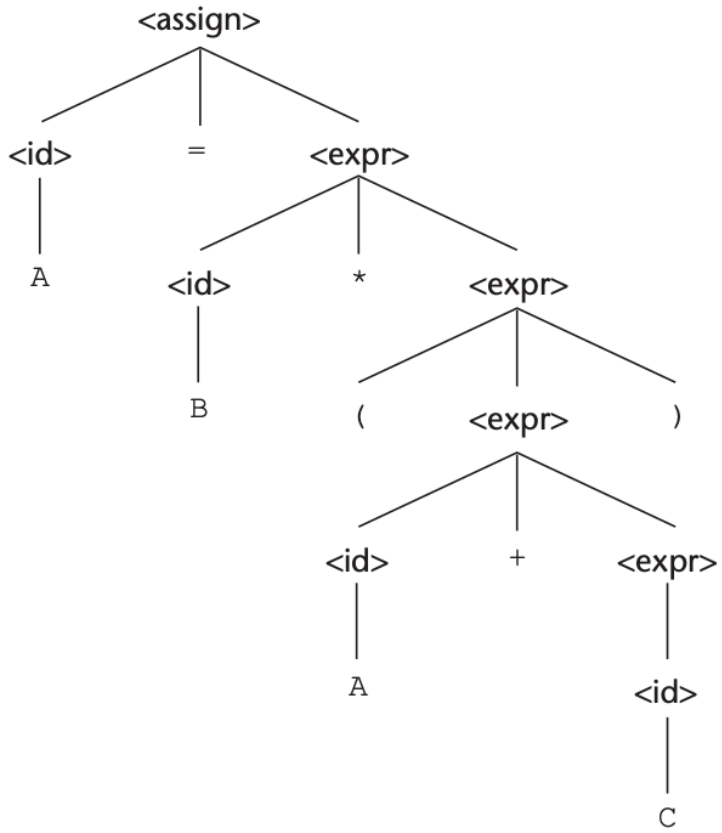
Parse Tree

- Parse trees are essential for visualising the derivation and understanding the hierarchical syntactic structure of the sentences of the language.
  - Root is the start symbol,
  - Internal nodes are non-terminals,
  - Leaf nodes are terminals.

Parse Tree

```
A = B * ( A + C )
```

<assign> => <id> = <expr>
=> A = <expr>
=> A = <id> * <expr>
=> A = B * <expr>
=> A = B * (<expr>)
=> A = B * (<id> + <expr>)
=> A = B * (A + <expr>)
=> A = B * (A + <id>)
=> A = B * (A + C)



Concepts of Programming Languages, Robert W. Sebesta

Parse Tree - Example

<program> → **begin** <stmt_list> **end**

<stmt_list> → <stmt>
| <stmt> ; <stmt_list>

<stmt> → <var> = <expression>

<var> → A | B | C

<expression> → <var> + <var>
| <var> – <var>
| <var>

```
A = B + C ; B = C
```

Concepts of Programming Languages, Robert W. Sebesta

Parse Tree - Example

<program> → **begin** <stmt_list> **end**

<stmt_list> → <stmt>
| <stmt> ; <stmt_list>

<stmt> → <var> = <expression>

<var> → A | B | C

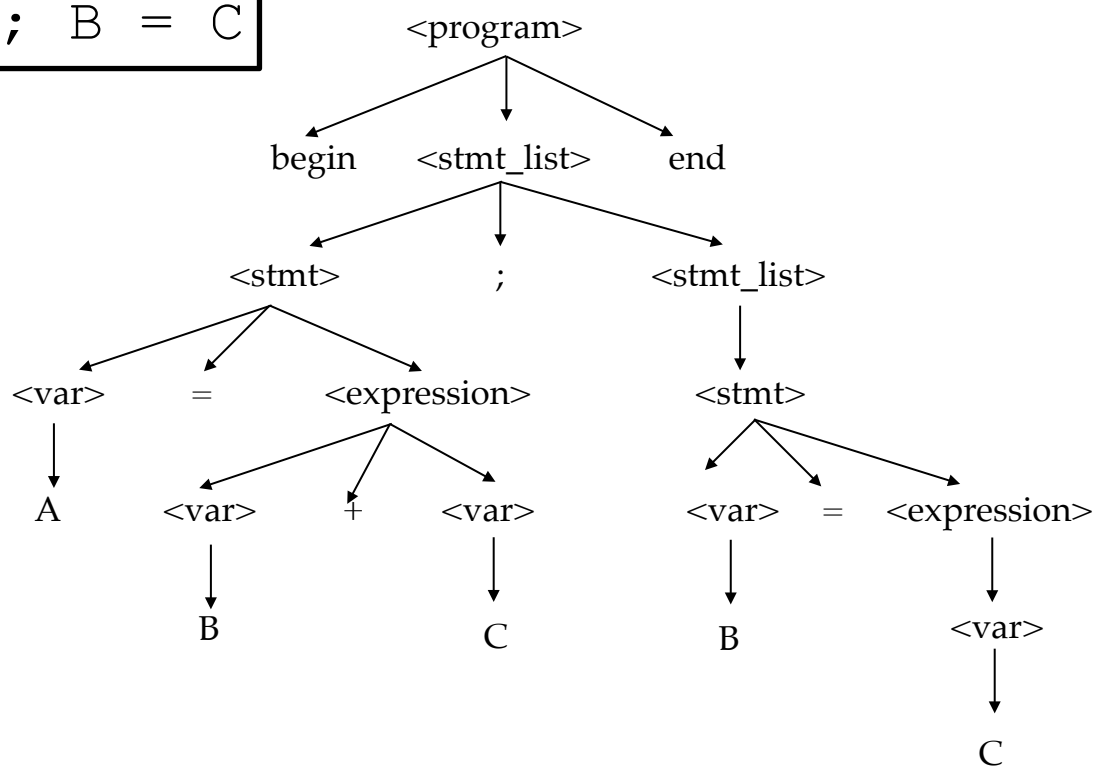
<expression> → <var> + <var>
| <var> – <var>
| <var>

```
A = B + C ; B = C
```

<program> => begin <stmt_list> end
=> begin <stmt> ; <stmt_list> end
=> begin <var> = <expression> ; <stmt_list> end
=> begin A = <expression> ; <stmt_list> end
=> begin A = <var> + <var> ; <stmt_list> end
=> begin A = B + <var> ; <stmt_list> end
=> begin A = B + C ; <stmt_list> end
=> begin A = B + C ; <stmt> end
=> begin A = B + C ; <var> = <expression> end
=> begin A = B + C ; B = <expression> end
=> begin A = B + C ; B = <var> end
=> begin A = B + C ; B = C end

Parse Tree - Example

```
A = B + C ; B = C
```

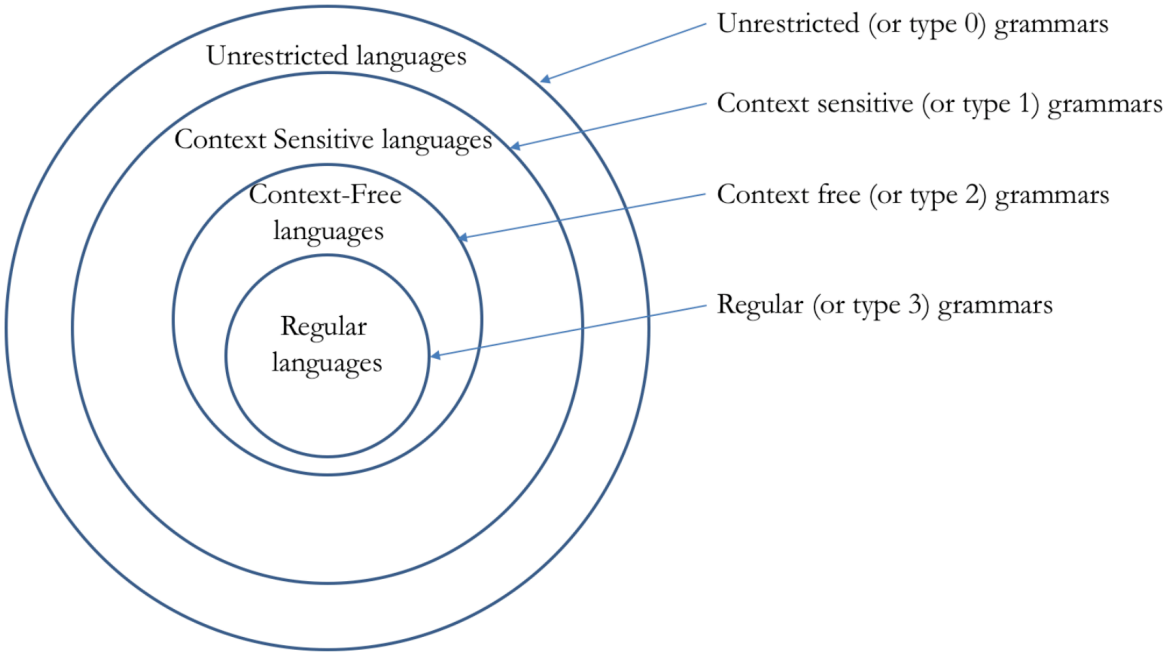


What is Context Free Grammar (CFG)

- Noam Chomsky
- 97 years old (1928)
- Doctorate degree in linguistics at the University of Pennsylvania
- Professor at MIT, linguistic

What is Context Free Grammar (CFG)

- Chomsky hierarchy
- Grammar is divided into 4 types.



https://www.taalhammer.com/the-chomsky-hierarchy-and-normative-grammar/

What is Context Free Grammar (CFG)

- Type 0: Unrestricted Grammar
  - Language recognised by the **Turing Machine**
- Type 1: Context-Sensitive Grammar
  - Languages recognised by **Linear Bound Automata**
- Type 2: Context-Free Grammar
  - Languages recognised by **Pushdown Automata**
- Type 3: Regular Grammar
  - Languages recognised by **Finite Automata**

What is Context Free Grammar (CFG)

- Type 0: Unrestricted Grammar
  - Language recognised by the **Turing Machine**

$$\alpha \rightarrow \beta$$

  - Allowing any string on the left (non-empty) to be replaced by any string on the right

What is Context Free Grammar (CFG)

- Type 1: Context-Sensitive Grammar
  - Languages recognised by **Linear Bound Automata**
  - $A \rightarrow B$ only if A is in context $(\alpha A \beta \rightarrow \alpha B \beta)$
    - (α, β) Context strings (can be empty, made of terminals/non-terminals).
  - A symbol can only be replaced if it appears in a specific surrounding sequence of symbols.

What is Context Free Grammar (CFG)

- Type 2: Context-Free Grammar
  - Terminal symbols (Σ)
  - Nonterminal symbols (V)
  - Production rules (P) $A \rightarrow a$
  - Start symbol (S) $Where\ a = \{V \cup \Sigma\}^*\ and\ A \in V$
  - $G = \{V, \Sigma, S, P\}$

$$G = \{(S, A), (a, b), S, (S \rightarrow aAb, A \rightarrow aAb \mid \in)\}$$

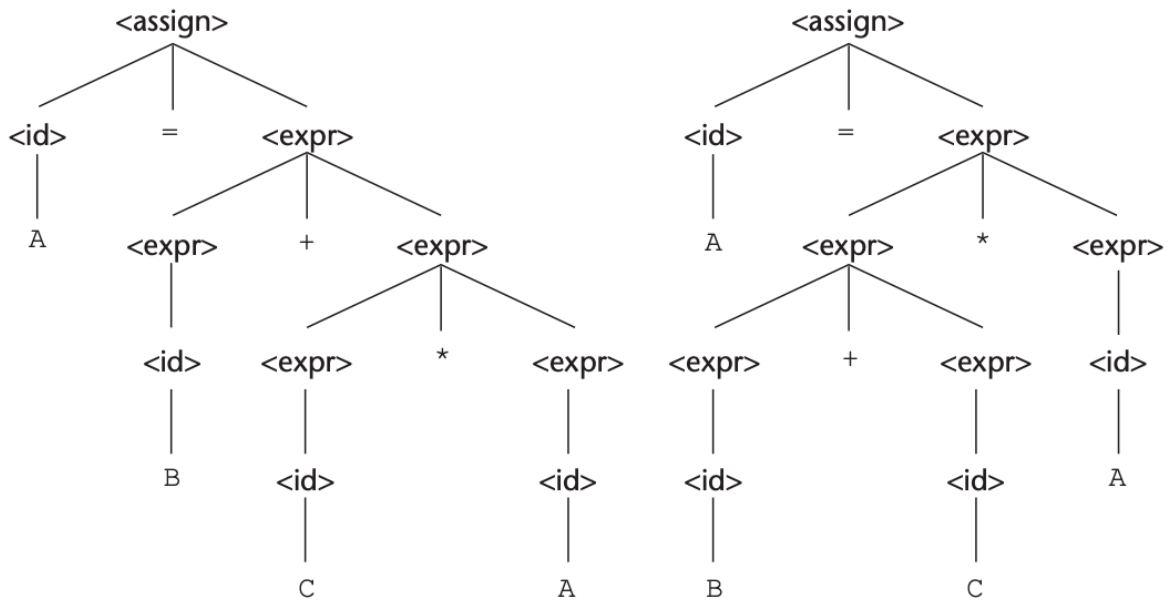
What is Context Free Grammar (CFG)

- Type 3: Regular Grammar
  - Languages recognised by **Finite Automata**
  - Right-linear: $A \rightarrow \alpha B$ or $A \rightarrow \alpha$ or $A \rightarrow \epsilon$ (empty string)
  - Left-linear: $A \rightarrow B\alpha$ or $A \rightarrow \alpha$ or $A \rightarrow \epsilon$ (non-terminals at the start)
  - Productions have at most one non-terminal on the right side, always at the end (right-linear) or beginning (left-linear)

Ambiguity

- Two or more distinct parse trees

```
A = B + C * A
```



Concepts of Programming Languages, Robert W. Sebesta

Ambiguity

- Problems when a single piece of code can have multiple valid parse trees,
  - Undefined Semantics and Inconsistent Behaviour
  - Difficulty in Parsing
  - The Dangling Else Problem
  - Developer Confusion

Operator Precedence

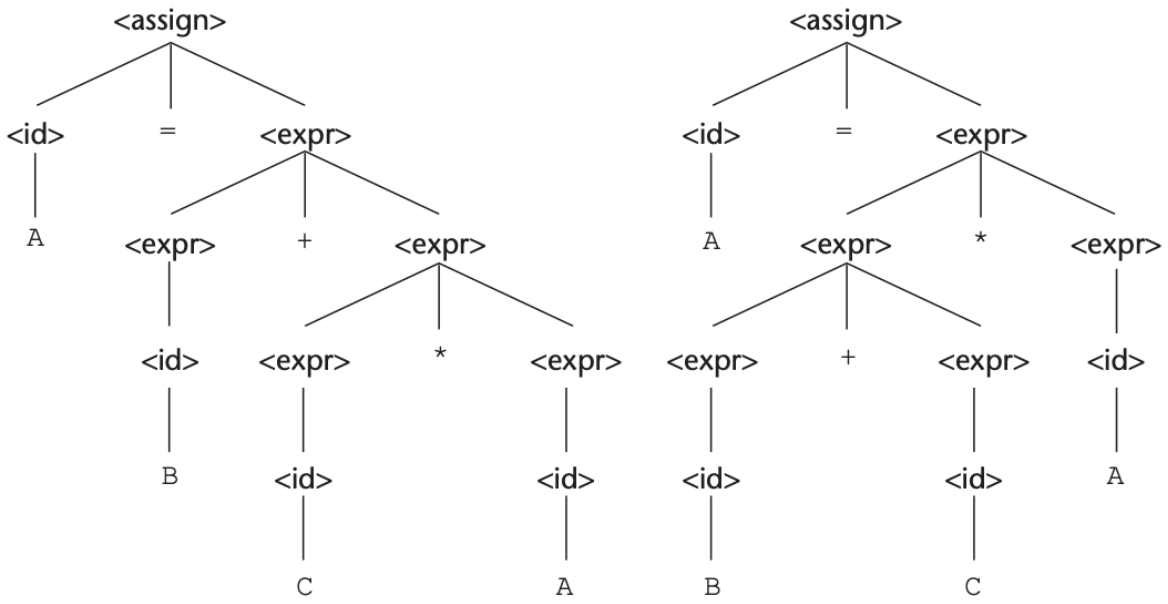
- Construct the parse tree

```
A = B + C * A
```

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <id> + <expr>
| <id> * <expr>
| (<expr>)
| <id>

Operator Precedence

```
A = B + C * A
```

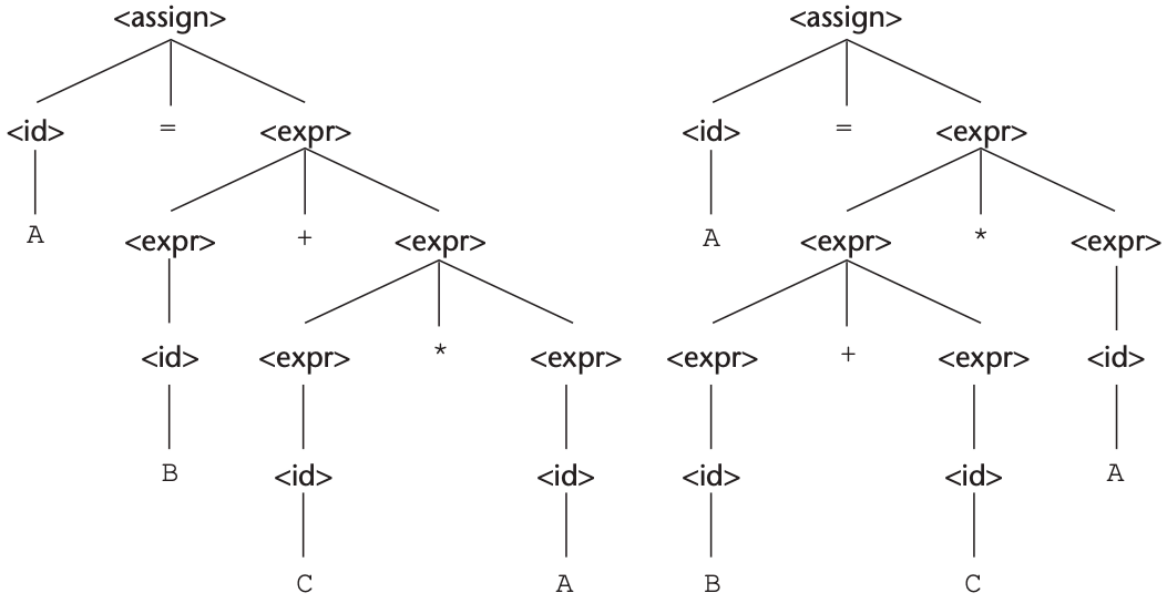


Concepts of Programming Languages, Robert W. Sebesta

Operator Precedence

```
A = B + C * A
```

Can you think of a solution for this problem?



Operator Precedence

- Instead of using <expr> for both operands of both + and *, define a different non-terminal.

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <term>
| <term>
<term> → <term> * <factor>
| <factor>
<factor> → (<expr>)
| <id>

<assign> → <id> = <expr>
<id> → A| B | C
<expr> → <id> + <expr>
| <id> * <expr>
| (<expr>)
| <id>

Operator Precedence

```
A = B + C * A
```

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <term>
| <term>
<term> → <term> * <factor>
| <factor>
<factor> → (<expr>)
| <id>

Try this.

Operator Precedence

```
<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <term>
         | <term>
<term> → <term> * <factor>
         | <factor>
<factor> → ( <expr>)
           | <id>
```

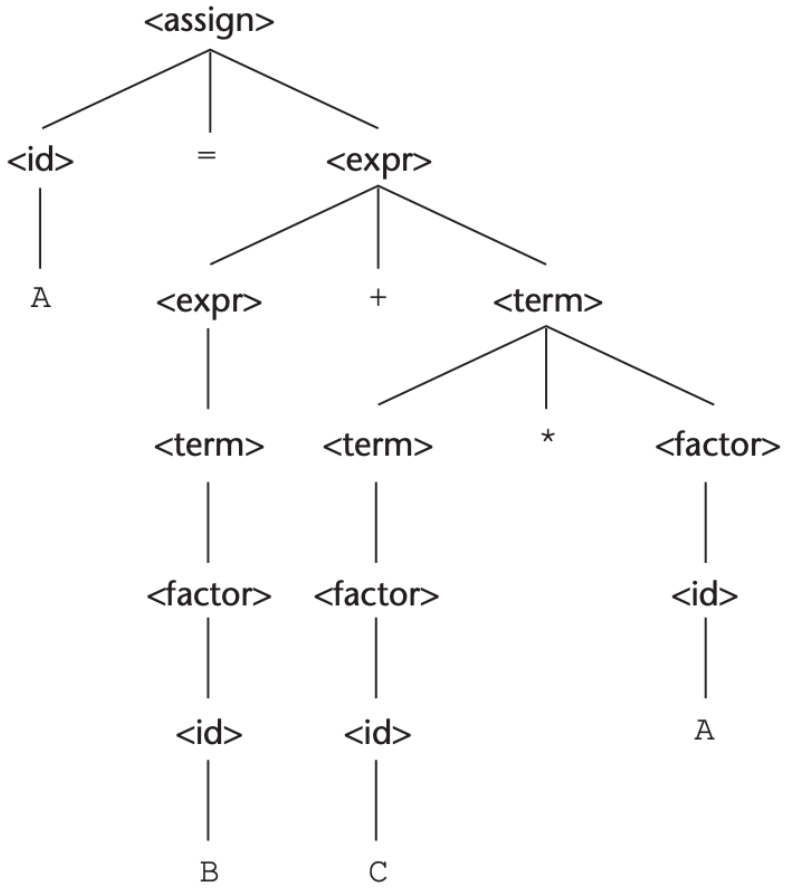
```
A = B + C * A
```

```
<assign> => <id> = <expr>
         => A = <expr>
         => A = <expr> + <term>
         => A = <term> + <term>
         => A = <factor> + <term>
         => A = <id> + <term>
         => A = B + <term>
         => A = B + <term> * <factor>
         => A = B + <factor> * <factor>
         => A = B + <id> * <factor>
         => A = B + C * <factor>
         => A = B + C * <id>
         => A = B + C * A
```

```
<assign> => <id> = <expr>
         => <id> = <expr> + <term>
         => <id> = <expr> + <term> * <factor>
         => <id> = <expr> + <term> * <id>
         => <id> = <expr> + <term> * A
         => <id> = <expr> + <factor> * A
         => <id> = <expr> + <id> * A
         => <id> = <expr> + C * A
         => <id> = <term> + C * A
         => <id> = <factor> + C * A
         => <id> = <id> + C * A
         => <id> = B + C * A
         => A = B + C * A
```

Operator Precedence

```
A = B + C * A
```



Concepts of Programming Languages, Robert W. Sebesta

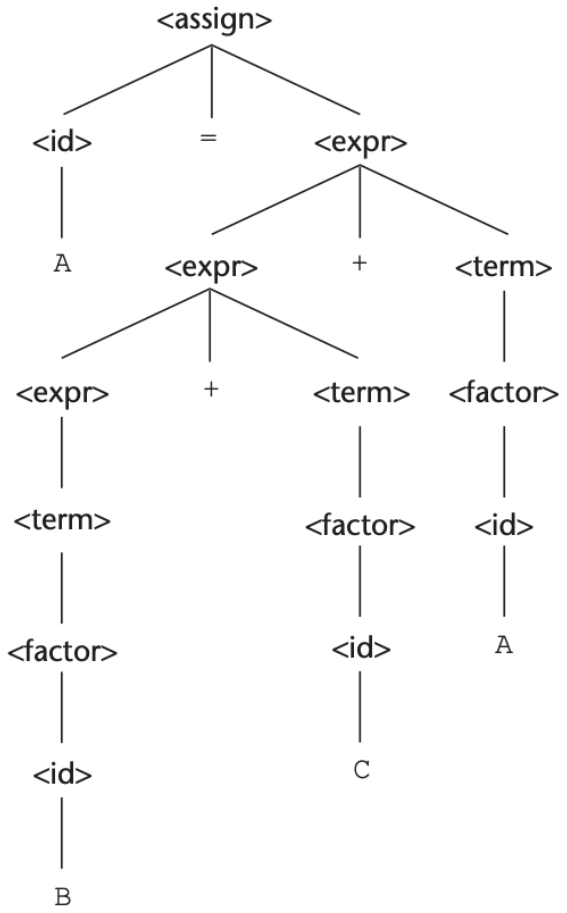
Associativity of Operators

- Two or more operators with the same precedence in an expression
  - Left-to-right associativity
    - 10 - 5 - 2
    - (10 - 5) - 2
  - Right-to-left associativity
    - a = b = 5
    - a = (b = 5)

Associativity of Operators

- A = B + C + A

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <term>
| <term>
<term> → <term> * <factor>
| <factor>
<factor> → (<expr>)
| <id>



Concepts of Programming Languages, Robert W. Sebesta

Associativity of Operators

Operator	Description	Associativity	Example
^	Exponential	Right to left	a^b
* /	Multiplication and division	Left to right	a*b
+ -	Addition and subtraction	Left to right	A-b

Associativity of Operators

<assign> → <id> = <expr>
<expr> → <expr> + <term> | <expr> - <term> | <term>
<term> → <term> * <factor> | <term> / <factor> | <factor>
<factor> → <newexp> ^ <factor> | <newexp>
<newexp> → (<expr>) | <id>
<id> → A | B | C

- Create parse trees for,
  - A = B + C + A
  - A = B ^ C ^ A

Try this.

Left Recursion and Right Recursion

- Left Recursion - non-terminal symbol directly or indirectly defines itself as the very first symbol in its own production rule.

$$A \rightarrow Ax \mid y$$

- Direct Left Recursion - non-terminal directly refers to itself at the beginning of its production

$$Expr \rightarrow Expr + Term$$

- Indirect Left Recursion - series of production rules eventually lead back to the original non-terminal

$$A \rightarrow Bx,\ B \rightarrow Cy,\ C \rightarrow Az$$

Left Recursion and Right Recursion

- Right Recursion

$$A \rightarrow xA \mid y$$

- Fit for top-down parsers, which allows the parser to consume the first part of the rule before making a recursive call.
- Problematic for left associative operators like subtraction. Eg: 5 - 3 - 2 should be evaluated as (5 - 3) - 2, not 5 - (3 - 2).

Problem in `if` statement

```
if (conditionA)
if (conditionB)
    statement1;
else
    statement2;
```

Problem in `if` statement

```
if (conditionA){
    if (conditionB)
        statement1;
    else
        statement2;
}
```

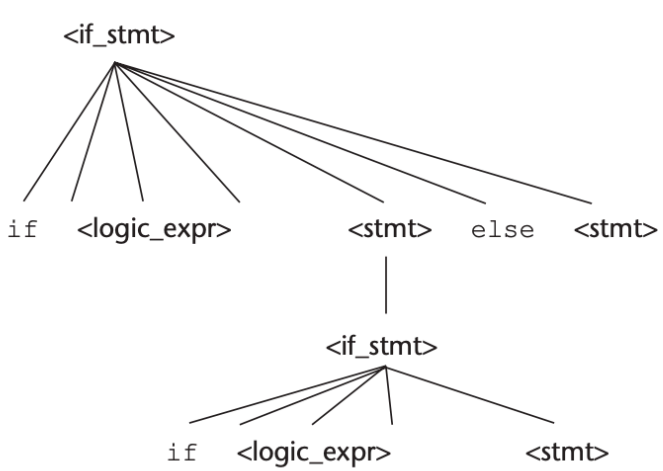
```
if (conditionA){
    if (conditionB)
        statement1;
}else{
    statement2;
}
```

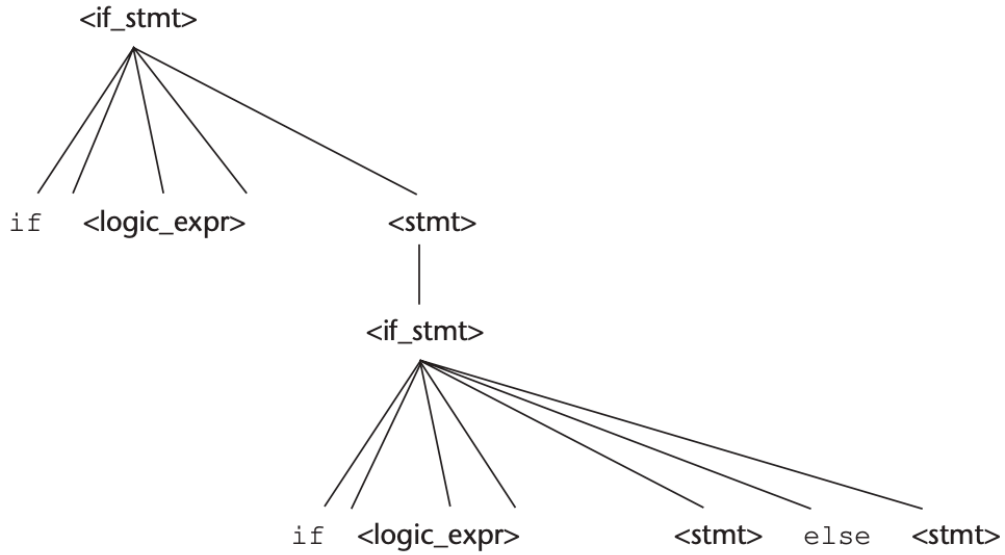
Problem in `if` statement

<if_stmt> → **`if`** (<logic_expr>) <stmt>

`if` (<logic_expr>) <stmt> **`else`** <stmt>

if (<logic_expr>) if (<logic_expr>) <stmt> else <stmt>





Problem in `if` statement

- Solve the problem
  - The "Nearest-If" Rule - an else always associates with the closest preceding if that does not already have an else.
  - Explicit Delimitation - clear end to an if statement block (end if or fi)

Problem in `if` statement

<stmt> → <matched> | <unmatched>
<matched> → **if** (<logic_expr>) <matched> **else** <matched>
| any non-if statement
<unmatched> → **if** (<logic_expr>) <stmt>
| **if** (<logic_expr>) <matched> **else** <unmatched>

- Construct the parse tree for the following

if (<logic_expr>) **if** (<logic_expr>) <stmt> **else** <stmt>

Extended BNF

- Increase the readability and writability.
  - [ ] for Optional Elements

<if_stmt> → **if** (<expression>) <statement>
| **if** (<expression>) <statement> **else** <statement>

<if_stmt> → **if** (<expression>) <statement> [**else** <statement>]

Extended BNF

- Increase the readability and writability.
  - { } for Repetition

<list> → <item> | <list>, <item>

<list> → <item> { , <item> }

  - ( ) for Grouping

<term> → <term> * <factor>
| <term> / <factor>
| <term> % <factor>

<term> → <term> (* | / | %) <factor>

Extended BNF

BNF and EBNF Versions of an Expression Grammar

BNF:

```
<expr> → <expr> + <term>
       | <expr> - <term>
       | <term>
<term> → <term> * <factor>
       | <term> / <factor>
       | <factor>
<factor> → <exp> ** <factor>
         | <exp>
<exp> → (<expr>)
      | id
```

EBNF:

```
<expr> → <term> {(+ | -) <term>}
<term> → <factor> {(* | /) <factor>}
<factor> → <exp> { ** <exp>}
<exp> → (<expr>)
      | id
```

Static Semantics
Attribute Grammars

- Extension to the context free grammars.
- Add semantic information (meaning and **context-sensitive** rules) to a context-free grammar.
- Adding attributes to the grammar symbol (Data values (e.g., int, float, string)).
- Use for type checking, expression evaluation.

```
A = B * ( A + C )
```

Attribute Grammars

- Attributes are categorised based on how their values are computed
  - Synthesised Attributes (Bottom-Up Flow)
    - Value of the parent node is synthesised from its children
    - Information flows up the parse tree, from the leaves towards the root
  - Inherited Attributes (Top-Down)
    - Attribute at a node is computed from the attribute values of its parent node and/or its siblings
    - Information flows down the parse tree from parent to child, or across from left siblings to right siblings

Attribute Grammars

- Expected Type (Inherited Attribute)
  - From the parent node
- Actual Type (Synthesised Attribute)
  - Flows up from the leaves to the parent node

Attribute Grammars

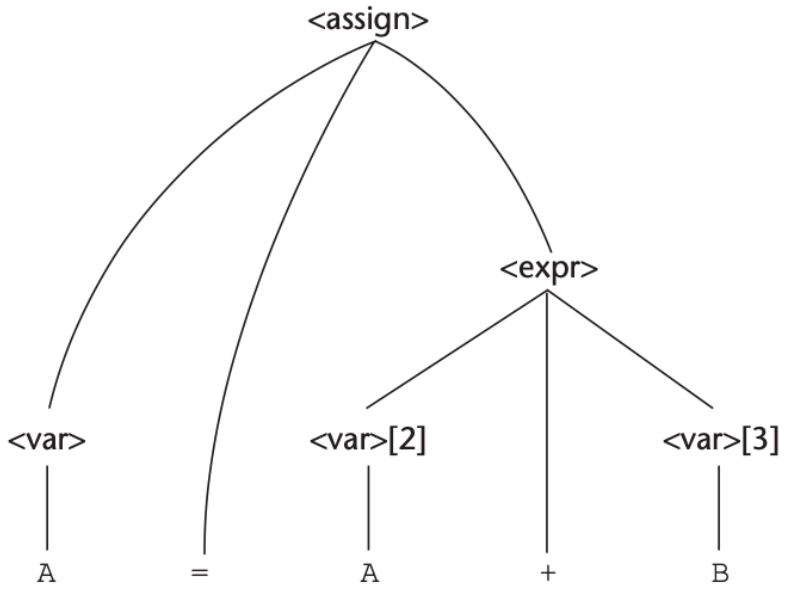
<assign> → <var> = <expr>
<expr> → <var> + <var>
| <var>
<var> → A | B | C

$$A = A + B$$

Attribute Grammars

<assign> → <var> = <expr>
<expr> → <var> + <var>
| <var>
<var> → A | B | C

A = A + B



Attribute Grammars

<assign> → <var> = <expr>
<expr> → <var> + <var>
| <var>
<var> → A | B | C

An Attribute Grammar for Simple Assignment Statements

1. Syntax rule: <assign> → <var> = <expr>
   Semantic rule: <expr>.expected_type ← <var>.actual_type
2. Syntax rule: <expr> → <var>[2] + <var>[3]
   Semantic rule: <expr>.actual_type ←
   if (<var>[2].actual_type = int) and
   (<var>[3].actual_type = int)
   then int
   else real
   end if
   Predicate: <expr>.actual_type == <expr>.expected_type
3. Syntax rule: <expr> → <var>
   Semantic rule: <expr>.actual_type ← <var>.actual_type
   Predicate: <expr>.actual_type == <expr>.expected_type
4. Syntax rule: <var> → A | B | C
   Semantic rule: <var>.actual_type ← look-up (<var>.string)

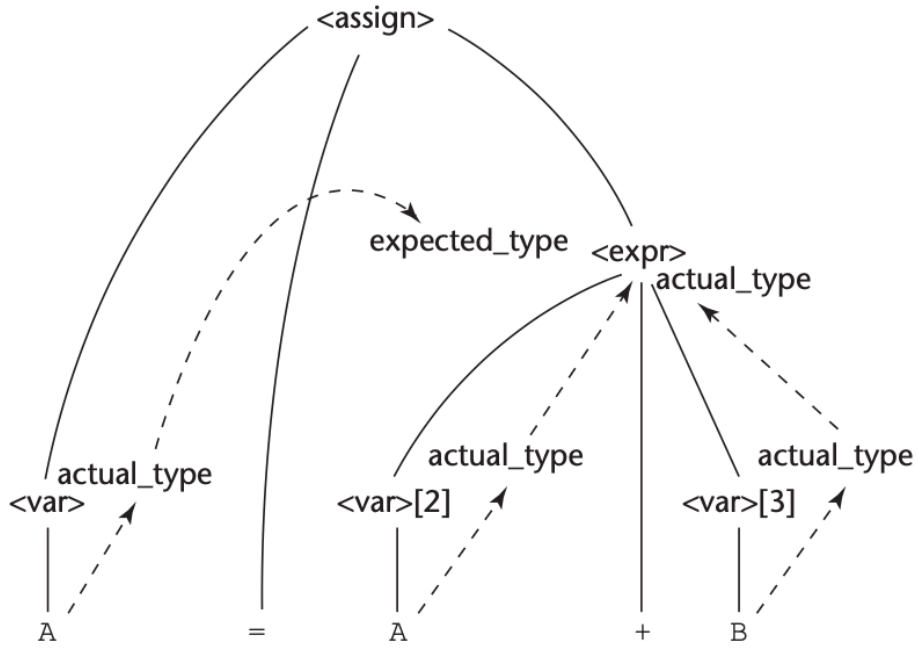
The look-up function looks up a given variable name in the symbol table and returns the variable's type.

Attribute Grammars

An Attribute Grammar for Simple Assignment Statements

1. Syntax rule: <assign> → <var> = <expr>
   Semantic rule: <expr>.expected_type ← <var>.actual_type
2. Syntax rule: <expr> → <var>[2] + <var>[3]
   Semantic rule: <expr>.actual_type ←
   if (<var>[2].actual_type = int) and
   (<var>[3].actual_type = int)
   then int
   else real
   end if
   Predicate: <expr>.actual_type == <expr>.expected_type
3. Syntax rule: <expr> → <var>
   Semantic rule: <expr>.actual_type ← <var>.actual_type
   Predicate: <expr>.actual_type == <expr>.expected_type
4. Syntax rule: <var> → A | B | C
   Semantic rule: <var>.actual_type ← look-up (<var>.string)

The look-up function looks up a given variable name in the symbol table and returns the variable's type.

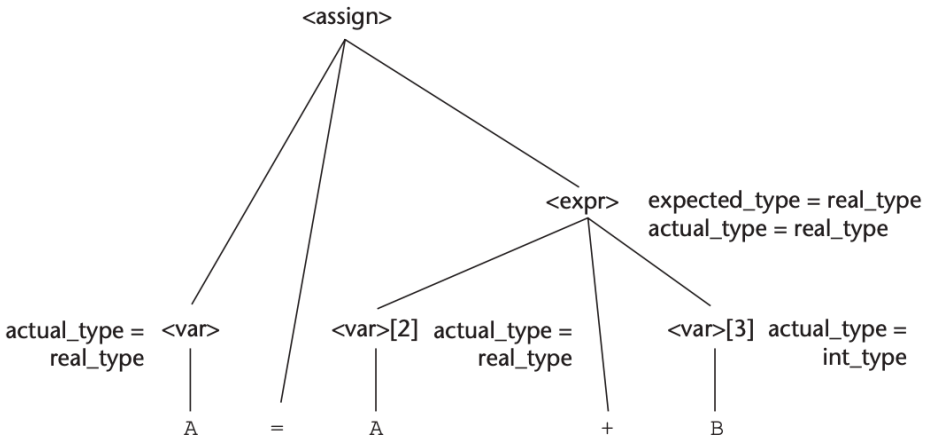


Attribute Grammars

An Attribute Grammar for Simple Assignment Statements

1. Syntax rule: <assign> → <var> = <expr>
   Semantic rule: <expr>.expected_type ← <var>.actual_type
2. Syntax rule: <expr> → <var>[2] + <var>[3]
   Semantic rule: <expr>.actual_type ←
   if (<var>[2].actual_type = int) and
   (<var>[3].actual_type = int)
   then int
   else real
   end if
   Predicate: <expr>.actual_type == <expr>.expected_type
3. Syntax rule: <expr> → <var>
   Semantic rule: <expr>.actual_type ← <var>.actual_type
   Predicate: <expr>.actual_type == <expr>.expected_type
4. Syntax rule: <var> → A | B | C
   Semantic rule: <var>.actual_type ← look-up (<var>.string)

The look-up function looks up a given variable name in the symbol table and returns the variable's type.



A is real

B is int

Attribute Grammars

An Attribute Grammar for Simple Assignment Statements

1. Syntax rule: <assign> → <var> = <expr>
   Semantic rule: <expr>.expected_type ← <var>.actual_type
2. Syntax rule: <expr> → <var>[2] + <var>[3]
   Semantic rule: <expr>.actual_type ←
   if (<var>[2].actual_type = int) and
   (<var>[3].actual_type = int)
   then int
   else real
   end if
   Predicate: <expr>.actual_type == <expr>.expected_type
3. Syntax rule: <expr> → <var>
   Semantic rule: <expr>.actual_type ← <var>.actual_type
   Predicate: <expr>.actual_type == <expr>.expected_type
4. Syntax rule: <var> → A | B | C
   Semantic rule: <var>.actual_type ← look-up (<var>.string)

The look-up function looks up a given variable name in the symbol table and returns the variable's type.

```
A = B + C * A
```

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <term>
| <term>
<term> → <term> * <factor>
| <factor>
<factor> → (<expr>)
| <id>

Try this.

A = 10 + 5 * 1.5

Dynamic Semantics

- Static Vs Dynamic semantics
  - Static (Legal Rules)
    - Compile-time, Type checking, variable declaration checks, scope rules
    - Attribute Grammars
  - Dynamic (Actual Meaning)
    - Runtime, Variable assignment results, loop behaviour, pointer dereferencing
    - Operational, Denotational, or Axiomatic semantics

Dynamic Semantics - Operational

- Dynamic semantics describes what a program does when it executes
- This model program execution as a sequence of state transformations
  - Program State (values of all variables, the program counter etc)
  - Transition Rules - specify how a program state changes when a particular statement is executed

Dynamic Semantics - Operational

C Statement

```
for (expr1; expr2; expr3) {
   ...
}
```

Meaning

```
        expr1;
loop:   if expr2 == 0 goto out
        ...
        expr3;
        goto loop
out:    ...
```

Dynamic Semantics - Operational

- Simple Example

```
x = y / z
```

- Static semantics - are `x, y, z` declared. Types of `x, y, z`. Etc
- Dynamic semantics - Take the current value of `y`, divide it by the current value of `z`, and update the memory location of x with the result. If `z` is 0, trigger an exception.

Dynamic Semantics - Axiomatic

- Defines the meaning of a program in terms of logical assertions.
- It describes what conditions must be true before and after a statement's execution.

$$\{P\}S\{Q\}$$

  - P (Precondition): What must be true before S executes.
  - S: The program statement.
  - Q (Postcondition): What is guaranteed to be true after S executes

Dynamic Semantics - Denotational

- Defines the meaning of a program by mapping it to a mathematical function